DESIGN PATTERNS

PATRONS DE CONCEPTION

PATRONS DE CONCEPTION

Introduction

Un patron de conception (souvent appelé design pattern) est un arrangement caractéristique de modules, reconnu comme bonne pratique en réponse à un problème de conception d'un logiciel. Il décrit une solution standard, utilisable dans la conception de différents logiciels
- Wikipedia

En d'autres termes : ce sont des manières classiques et courantes de résoudre des problèmes communs en programmation

Sont ajoutés à cette listes des notions qui ne sont pas des *design patterns*, mais qui s'intègrent bien à cette partie du cours.

PATRONS DE CONCEPTION

Le programme

- Les classes abstraites
- Les éléments statiques
- Singleton
- Builder
- Adapter
- Decorator
- Model - Vue - Controller

Il en existe plein d'autres, que nous ne verrons pas dans ce cours.

Voir https://en.wikipedia.org/wiki/Software_design_pattern

PATRONS DE CONCEPTION

Les classes abstraites

Une classe abstraite est une classe qui ne peut pas être instanciée.

Elle ne sera utilisé que comme parent d'une autre classe.

Cela permet de définir des ensemble de méthodes communes à plusieurs classes.

PATRONS DE CONCEPTION

Les classes abstraites

```
// Classe abstraite, non instanciable
class Storage {
  constructor() {
    if (new.target === Storage) {
      throw new Error("Storage is abstract");
    }
  }

  // Méthode abstraite, non implémentée
  write(key, value) {
    throw new Error("write is not implemented");
  }

  load(key) {
    throw new Error("load is not implemented");
  }

  exists(key) {
    const value = this.load(key);
    return value !== null && value !== undefined;
  }
}
```

```
class LocalStorage extends Storage {
  constructor() {
    super();
  }

  write(key, value) { /* sauvegarde locale */ }

  load(key) { /* récupère la valeur locale */ }
}


class DistantStorage extends Storage {
  constructor() {
    super();
  }

  write(key, value) { /* sauvegarde distante */ }

  load(key) { /* récupère la valeur distante */ }
}
```

PATRONS DE CONCEPTION

Les classes abstraites

A utiliser quand

- Plusieurs classes ont un ensemble commun de méthodes groupables
- Cet ensemble n'est pas suffisant pour créer un objet complet

A ne pas utiliser quand

- On pourrait avoir une classe parent non abstraite
- Grouper les méthodes communes ne forme pas une classe cohérente

PATRONS DE CONCEPTION

Les éléments statiques

Parfois, il est utile de placer des valeurs ou des fonctions dans des objets, mais qui ne sont pas liés à une instance. C'est le cas de la classe `Math` :

```
const area = Math.PI * 3 * 3;
const cosinus = Math.cos(12);
const absolute = Math.abs(-45);
const euler_number = Math.E;
```

Math est une classe qui n'est jamais instanciée. A aucun moment on ne fait `const value = new Math();`

Tous les éléments de `Math` sont dit *statiques*. Les éléments statiques sont simplement "rangés" dans une classe.

PATRONS DE CONCEPTION

Les éléments statiques

Les éléments statiques sont indiqués par le mot clé `static`.

```
class MyMath {
  static PI = 3.141592653589793;
  static E = 2.718281828459045;

  static add(a, b) {
    return a + b;
  }

  static square(x) {
    return x * x;
  }
}

console.log(MyMath.PI); // 3.1415....
console.log(MyMath.add(3,5)); // 8
```

On peut également empêcher l'instanciation de la même manière que pour une classe abstraite.

Il est possible de combiner éléments statiques et non statiques dans une même classe.

PATRONS DE CONCEPTION

Les éléments statiques

A utiliser quand

- On a besoin de grouper des fonctions / valeurs
- On a un attribut ou une méthode d'une classe ne nécessitant pas une instance de la classe
  Par exemple : constantes, opérations ne nécessitant aucun attribut, etc

A ne pas utiliser quand

- Une valeur statique peut changer, il faut alors en faire un attribut
- Une fonction statique utilise des attributs, il faut alors en faire une méthode

PATRONS DE CONCEPTION

Singleton

Un *Singleton* est un objet ne pouvant être instancié qu'une fois.

Il est global à toute l'exécution du programme.

Créer un deuxième objet peut soit retourner une erreur, soit retourner le même objet global.

PATRONS DE CONCEPTION

Singleton

```
class ConfigManager {
  constructor() {
    if (ConfigManager.instance) {
      return ConfigManager.instance;
    }

    this.config = {};
    ConfigManager.instance = this;
  }

  set(key, value) {
    this.config[key] = value;
  }

  get(key) {
    return this.config[key];
  }
}
```

```
// Dans un module A
const configA = new ConfigManager();
configA.set("fontSize", 10);
```

```
// Dans un module B
// ConfigManager est un Singleton, il n'en existe qu'un
const configB = new ConfigManager();
console.log(configB.get("fontSize")); // 10
```

Il existe plusieurs solutions d'implémentation. On pourrait aussi utiliser une fonction statique `ConfigManager.getInstance()` pour obtenir l'objet.

PATRONS DE CONCEPTION

Singleton

A utiliser quand

- On a besoin d'un objet unique et partagé partout dans notre programme
- Il ne fait aucun sens d'avoir plusieurs objet de ce type

A ne pas utiliser quand

- On souhaite créer une valeur globale, facilement accessible. C'est souvent une fausse bonne idée.
- On pourrait avoir besoin de plusieurs instances de la classe

Attention : Le singleton est un pattern souvent critiqué, dont il est facile d'abuser.
On a tendance à l'utiliser pour se simplifier la vie, alors qu'il crée de grands effets de bord !

PATRONS DE CONCEPTION

Builder

Certains objets peuvent devenir complexes à construire, avec des constructeurs possédant beaucoup de paramètres.
Dans d'autres cas, il est difficile de construire un objet en une fois, à un seul endroit dans le code.

Le pattern *Builder* vise à construire un objet final en plusieurs étapes.

PATRONS DE CONCEPTION

Builder

```
// Classe décrivant une requête SQL
class Query {
  constructor(select, from, where, order) {
    this.select = select;
    this.from = from;
    this.where = where;
    this.order = order;
  }

  toString() {
    return `SELECT ${this.select} FROM ${this.from} WHERE ${this.where} ORDER BY ${this.orde
  }
}
```

PATRONS DE CONCEPTION

Builder

```
class QueryBuilder {
  setSelect(fields) {
    this.select = fields;
    return this;
  }
  setFrom(table) {
    this.from = table;
    return this;
  }
  setWhere(cond) {
    this.where = cond;
    return this;
  }
  setOrderBy(field) {
    this.orderBy = field;
    return this;
  }

  build() {
    return new Query(this.select, this.from, this.where, this.orderBy);
  }
}
```

PATRONS DE CONCEPTION

Builder

```
const query = new QueryBuilder()
  .setSelect("*")
  .setFrom("users")
  .setWhere("age > 18")
  .setOrderBy("name")
  .build();

console.log(query.toString());
```

PATRONS DE CONCEPTION

Builder

```
const qBuilder = new QueryBuilder()
/* ... */
qBuilder = qBuilder.setSelect("*")
/* ... */
qBuilder = qBuilder.setFrom("users")
/* ... */
qBuilder = qBuilder.setWhere("age > 18")
/* ... */
qBuilder = qBuilder.setOrderBy("name")
/* ... */
const query = qBuilder.build();
console.log(query.toString());
```

PATRONS DE CONCEPTION

Builder

A utiliser quand

- Un objet doit être construit en plusieurs étapes
- La construction d'un objet en une ligne devient trop complexe

A ne pas utiliser quand

- On pourrait simplifier autrement la construction de l'objet. Le pattern *Builder* apporte beaucoup de complexité
- Il y a trop de risques que le *Builder* se trouve dans des états invalides

PATRONS DE CONCEPTION

Adapter

Un adapter permet de transformer l'interface d'une classe pour qu'elle "s'adapte" à un besoin donné

```
class KMLReader {
  constructor(filePath) {
    this.filePath = filePath;
  }

  readKML() { /* Lit le contenu du fichier */ }
}


class GeoJSONAdapter {
  constructor(kmlReader) {
    this.kmlReader = kmlReader;
  }

  readJson() { /* Utilise this.kmlReader pour produire du GeoJSON */ }
}
```

PATRONS DE CONCEPTION

Adapter

Un adapter permet de transformer l'interface d'une classe pour qu'elle "s'adapte" à un besoin donné

```
const kmlReader = new KMLReader("map.kml");
const geojsonAdapter = new GeoJSONAdapter(kmlReader);
const geojson = geojsonAdapter.readJson();
```

PATRONS DE CONCEPTION

Adapter

A utiliser quand

- Une classe de base doit être modifiée, mais il n'est pas possible de le faire (par exemple, issue d'une librairie)
- L'héritage n'est pas une bonne solution

A ne pas utiliser quand

- Modifier la classe de base est possible / acceptable
- L'héritage pourrait être utilisé à la place

PATRONS DE CONCEPTION

Decorator

Un *Decorator* prend le résultat d'une classe et vient y ajouter quelque chose de nouveau

PATRONS DE CONCEPTION

Decorator

```
class Text {
  constructor(content) { this.content = content; }
  render() { return this.content; }
}

class Bold {
  constructor(text) { this.text = text; }
  render() { return `<b>${this.text.render()}</b>`; }
}

class Italic {
  constructor(text) { this.text = text; }
  render() { return `<i>${this.text.render()}</i>`; }
}

class Underline {
  constructor(text) { this.text = text; }
  render() { return `<u>${this.text.render()}</u>`; }
}

const text = new Text("Hello world");
const formatted = new Underline( new Bold( new Italic(text) ) );
console.log(formatted.render());
```

PATRONS DE CONCEPTION

Decorator

A utiliser quand

- On veut ajouter une ou plusieurs "surcouches" à un objet donné
- Modifier la classe de base est impossible ou peu pratique

A ne pas utiliser quand

- On veut simplement ajouter une méthode à un objet
- Le *Decorator* ajoute de nouvelles fonction. Il faut alors utiliser l'héritage, ou un autre pattern.

PATRONS DE CONCEPTION

Model - Vue - Controller (MVC)

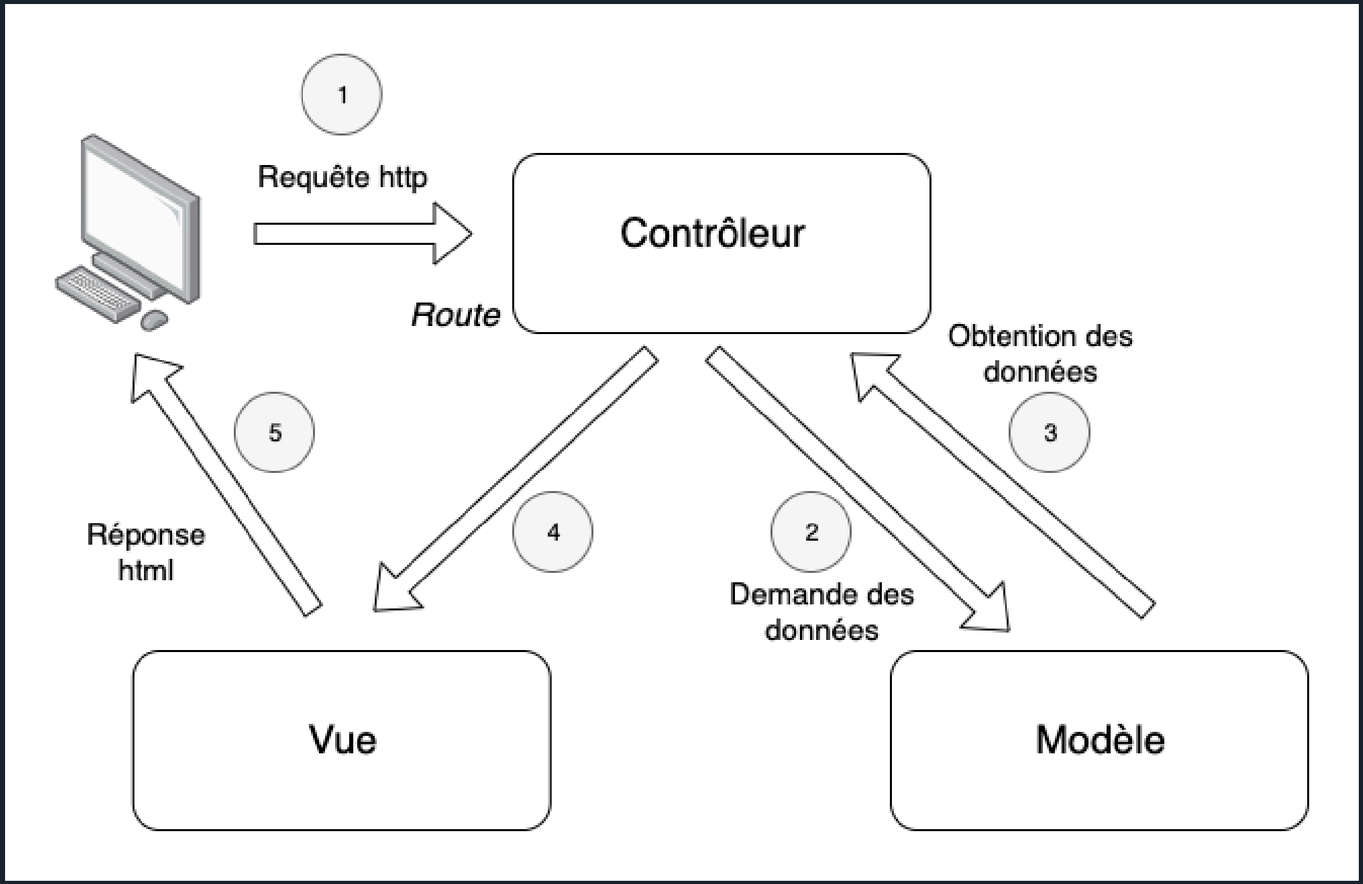
Tiré de Wikipédia :

Modèle-vue-contrôleur ou MVC est un motif d'architecture logicielle destiné aux interfaces graphiques, lancé en 1978 et très populaire pour les applications web. Le motif est composé de trois types de modules ayant trois responsabilités différentes : les modèles, les vues et les contrôleurs.

- Un modèle (Model) contient les données à afficher
- Une vue (View) contient la présentation de l'interface graphique
- Un contrôleur (Controller) contient la logique concernant les actions effectuées par l'utilisateur

PATRONS DE CONCEPTION

Model - Vue - Controller (MVC)



PATRONS DE CONCEPTION

Model - Vue - Controller (MVC)

- Le MVC peut être implémenté de nombreuses manières
- Il convient particulièrement bien aux applications web, avec frontend et backend
- De nombreux frameworks utilisent les principes du MVC : Ruby on Rails, Django, etc

PATRONS DE CONCEPTION

L'over-engineering

Quand faut-il utiliser des patrons de conception ?

Comment les utiliser efficacement ?

PATRONS DE CONCEPTION

L'over-engineering

Quand faut-il utiliser des patrons de conception ?

Comment les utiliser efficacement ?

La réponse est simple : K.I.S.S.

Autrement dit : seulement si cela rend le code plus clair et facile à comprendre.

Une erreur classique est la sur-utilisation de patterns : ce n'est pas parce qu'ils existent qu'il faut les utiliser.

Ce ne sont que des outils pour atteindre un code de qualité, par des garanties !

Voir par exemple le repo satirique https://github.com/EnterpriseQualityCoding/FizzBuzzEnterpriseEdition

Ce programme effectue un calcul très simple, mais noyé dans un code chaotique.